МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Курсовой проект
по курсу «Дискретный анализ»
на тему «F. Алгоритм BWT+MTF+RLE»

Выполнил: Н. В. Храмов
Группа: М8О-208БВ-24
Преподаватель: Макаров Н.К.
Проверил: Сахарин Н.А.
Дата: 05.11.2026
Оценка:
Подпись:

Москва, 2026

Постановка задачи

Тема F. «Алгоритм BWT+MTF+RLE». Требуется реализовать сжатие и восстановление текста посредством последовательного применения преобразования Барроуза — Уилера (BWT), перемещения в начало (MTF) и кодирования длин серий (RLE). Исходный текст состоит только из строчных латинских букв. Максимальная длина текста — 10^5 символов. Ограничение времени — 1 секунда, памяти — 256 МБ.

Для прямого BWT к тексту добавляется единственный символ $, который меньше любой буквы исходного алфавита. При восстановлении служебный символ удаляется. Начальное распределение кодов MTF фиксировано: $ → 0, a → 1, b → 2, …, z → 26. Пары RLE записываются в порядке «количество, значение».

Формат ввода

В режиме сжатия первая строка содержит команду compress, следующая — исходный текст. В режиме восстановления первая строка содержит decompress, далее до конца ввода перечисляются пары целых чисел, полученные при сжатии. Ввод осуществляется через стандартный поток; файл input.txt может быть подан перенаправлением.

Формат вывода

Для команды compress выводятся пары RLE, каждая на отдельной строке. Для команды decompress выводится восстановленный исходный текст без символа $. Результат записывается в стандартный поток; для сохранения в output.txt используется перенаправление.

Общая схема решения

Сжатие выполняется в порядке BWT → MTF → RLE. Восстановление проходит в обратном порядке: обратное RLE → обратное MTF → обратное BWT. Каждый этап является обратимым, поэтому цепочка сохраняет исходный текст без потерь. Уменьшение длины итогового представления для любой строки не гарантируется.

Алгоритм решения задачи

BWT заменяет строку последним столбцом лексикографически отсортированных циклических сдвигов. Одинаковые или близкие контексты оказываются рядом, что часто группирует предшествующие им символы. Полная матрица сдвигов не строится: каждый сдвиг представлен индексом начала.

Сортировка выполняется методом удвоения. Сначала позиции сортируются по символам и получают классы эквивалентности. На шаге с длиной k фрагмент длины 2k представляется парой (class[i], class[(i + k) mod n]). После сортировки пар назначаются новые классы, и длина удваивается.

Из отсортированных индексов вычитается k по модулю n: получаем порядок по вторым половинам. Устойчивая сортировка подсчётом по первой половине задаёт порядок пар за O(n). При уникальности классов порядок сдвигов определён. Символ перед началом каждого сдвига по кругу образует последний столбец BWT.

MTF поддерживает изменяемый список $abcdefghijklmnopqrstuvwxyz. Для каждого символа в результат записывается его текущая позиция, после чего символ переносится в начало списка. Повторение символа подряд после первого появления кодируется нулём. Недавно использованные символы остаются близко к началу списка.

RLE проходит по кодам MTF и объединяет соседние одинаковые значения. Новое значение создаёт пару с количеством 1; совпадение с последним значением увеличивает длину текущей серии. В данной реализации пары хранятся в структурах Run с полями count и value.

Обратные преобразования

Обратное RLE разворачивает каждую пару в указанное количество копий значения. Обратное MTF начинает с того же фиксированного списка: по коду извлекает символ и перемещает его в начало. Таким образом восстанавливается последний столбец BWT.

В обратном BWT частоты задают начала блоков символов в первом столбце. Вхождения одинаковых букв в последнем и первом столбцах связываются по порядку появления: строится LF. От позиции $ в последнем столбце выполняется обход LF с заполнением результата справа налево. Терминатор удаляется.

Доказательство корректности

Корректность сортировки BWT доказывается индукцией по длине фрагмента. На начальном шаге сортировка символов и классы длины 1 корректны. Если классы правильно упорядочивают фрагменты длины k, лексикографическое сравнение двух фрагментов длины 2k эквивалентно сравнению пар классов их половин. Предварительный сдвиг индексов упорядочивает вторые половины, а устойчивая сортировка по первым половинам сохраняет нужный порядок при равенстве первых классов. Следовательно, новые порядок и классы корректны.

При различных классах для всех позиций любой паре сдвигов уже соответствует различающийся просмотренный префикс. Поэтому продолжения не изменят результат сравнения. Если ранняя остановка не наступила, удвоение в итоге охватывает всю строку. Уникальный терминатор гарантирует различие всех полных сдвигов. Символ перед началом сдвига по кругу является его последним символом, следовательно, encode выдаёт требуемый последний столбец.

Для обратного BWT рассмотрим строки, заканчивающиеся одной и той же буквой. Их взаимный порядок определяется частями до последнего символа. При переносе общей последней буквы в начало порядок также определяется этими частями и сохраняется. Поэтому j-е вхождение буквы в последнем столбце соответствует j-му вхождению в первом. Построенный LF действительно ведёт к циклическому сдвигу с началом на один символ раньше.

Строка, содержащая $ в последнем столбце, является исходной строкой с терминатором. При движении по LF последовательно читаются её символы справа налево. После n переходов восстановлены все символы, включая единственный $. Удаление последнего символа возвращает исходный текст.

При кодировании и декодировании MTF списки первоначально одинаковы. Если они совпадают перед очередным шагом, записанный код однозначно указывает на исходный символ. Одинаковое перемещение этого символа в начало сохраняет равенство списков на следующем шаге. По индукции обратное MTF восстанавливает всю входную последовательность.

RLE разбивает последовательность на максимальные непрерывные серии равных значений. Пара сохраняет значение и точное число его вхождений. Разворачивание пар в исходном порядке восстанавливает все серии без изменений. Композиция трёх обратимых преобразований обратима, следовательно, программа правильно сжимает и восстанавливает допустимый текст.

Анализ сложности

Пусть m — длина исходного текста, n = m + 1 — длина с терминатором, r — число серий RLE, σ = 27 — размер алфавита MTF. Для корректного результата $1 \le r \le n$. Ниже память учитывает создаваемый результат, а время относится к операциям над данными в памяти.

Операция	**Время**	**Память**
Прямое BWT	O(n log n)	O(n)
Обратное BWT	O(n)	O(n)
Прямое и обратное MTF	O(nσ) = O(n)	O(n + σ) = O(n)
Прямое RLE	O(n)	O(r)
Обратное RLE	O(n + r) = O(n)	O(n)
Всё сжатие	O(n log n)	O(n)
Всё восстановление	O(n)	O(n)

Начальная сортировка BWT требует O(n + 256). Каждый шаг удвоения состоит из линейных проходов по индексам и классам. Таких шагов не более $\lceil \log_2 n \rceil$, поэтому время прямого BWT равно O(n log n). Пять рабочих векторов размера порядка n переиспользуются на каждом шаге: память не умножается на число итераций.

В обратном BWT частоты, начала блоков, LF и результат строятся за O(n + 256). Поскольку байтовый алфавит фиксирован, это O(n). В MTF поиск и перенос символа требуют до O(σ) операций, но σ постоянно. Без учёта результата дополнительная рабочая память MTF составляет O(σ), а RLE — O(1).

Этапы выполняются последовательно, поэтому времена складываются. При сжатии доминирует BWT, при восстановлении все этапы линейны. Промежуточные строки, коды и пары могут существовать одновременно, однако их суммарный размер остаётся O(n).

Консольная запись состоит из r строк с двумя десятичными числами. Её фактический размер зависит от числа серий и количества цифр в счётчиках. Поэтому оценку алгоритмических этапов следует отличать от измерений полного процесса с форматированием, чтением и записью.

Структура программы

BurrowsWheelerTransform предоставляет методы encode и decode. Первый реализует сортировку циклических сдвигов методом удвоения, второй — восстановление через LF. Все рабочие массивы локальны вызову, поэтому повторные обращения не зависят от предыдущего запуска.

MoveToFrontTransform отвечает за прямое и обратное MTF. Закрытый метод initialAlphabet создаёт начальный список символов, moveToFront выполняет перенос. RunLengthEncoding предоставляет кодирование и разворачивание серий. Структура Run хранит количество повторений и числовое значение.

TextCompressor объединяет три преобразования посредством композиции. Методы compress и decompress задают порядок этапов и возвращают результат. Функция main читает режим и данные, вызывает TextCompressor и форматирует вывод. Алгоритмы не зависят от стандартных потоков и доступны для отдельных измерений.

Разделение обязанностей обеспечивает инкапсуляцию и самостоятельность алгоритмов. Общие операции MTF вынесены в закрытые методы. Наследование не требуется: преобразования имеют разные типы входных и выходных данных и объединяются последовательностью вызовов. Методы объявлены const, поскольку состояние объектов между операциями не изменяется.

Исходный код

Ниже приведён полный исходный код main.cpp, использованный при измерениях. Сборка выполнялась в режиме C++23 с оптимизацией O2. Программа рассчитана на входные данные, удовлетворяющие условию задачи.

```
#include <algorithm>
#include <array>
#include <iostream>
#include <string>
#include <vector>

class BurrowsWheelerTransform {
public:
    std::string encode(const std::string& text) const {
        const std::string input = text + '$';
        const int n = static_cast<int>(input.size());
        std::vector<int> order(n), classes(n), shifted(n), nextClasses(n);
        std::vector<int> count(std::max(n, 256), 0);

        for (unsigned char ch : input) {
            ++count[ch];
        }
        for (int i = 1; i < 256; ++i) {
            count[i] += count[i - 1];
        }
        for (int i = n - 1; i >= 0; --i) {
            order[--count[static_cast<unsigned char>(input[i])]] = i;
        }

        int classCount = 1;
        classes[order[0]] = 0;
        for (int i = 1; i < n; ++i) {
            if (input[order[i]] != input[order[i - 1]]) {
                ++classCount;
            }
            classes[order[i]] = classCount - 1;
        }

        for (int length = 1; length < n && classCount < n; length *= 2) {
            for (int i = 0; i < n; ++i) {
                shifted[i] = order[i] - length;
                if (shifted[i] < 0) {
                    shifted[i] += n;
                }
            }
            std::fill(count.begin(), count.begin() + classCount, 0);
            for (int index : shifted) {
                ++count[classes[index]];
            }
            for (int i = 1; i < classCount; ++i) {
                count[i] += count[i - 1];
            }
            for (int i = n - 1; i >= 0; --i) {
                const int index = shifted[i];
                order[--count[classes[index]]] = index;
            }
```

```
            int nextClassCount = 1;
            nextClasses[order[0]] = 0;
            for (int i = 1; i < n; ++i) {
                const int current = order[i];
                const int previous = order[i - 1];
                if (classes[current] != classes[previous] ||
                    classes[(current + length) % n] !=
                        classes[(previous + length) % n]) {
                    ++nextClassCount;
                }
                nextClasses[current] = nextClassCount - 1;
            }
            classes.swap(nextClasses);
            classCount = nextClassCount;
        }

        std::string result(n, '\0');
        for (int i = 0; i < n; ++i) {
            result[i] = input[(order[i] + n - 1) % n];
        }
        return result;
    }

    std::string decode(const std::string& lastColumn) const {
        const int n = static_cast<int>(lastColumn.size());
        std::array<int, 256> frequency{};
        std::array<int, 256> start{};
        int row = -1;
        for (int i = 0; i < n; ++i) {
            ++frequency[static_cast<unsigned char>(lastColumn[i])];
            if (lastColumn[i] == '$') {
                row = i;
            }
        }
        for (int i = 1; i < 256; ++i) {
            start[i] = start[i - 1] + frequency[i - 1];
        }

        std::vector<int> lf(n);
        for (int i = 0; i < n; ++i) {
            const auto ch = static_cast<unsigned char>(lastColumn[i]);
            lf[i] = start[ch]++;
        }
        std::string result(n, '\0');
        for (int i = n - 1; i >= 0; --i) {
            result[i] = lastColumn[row];
            row = lf[row];
        }
        result.pop_back();
        return result;
    }
};

class MoveToFrontTransform {
public:
    std::vector<int> encode(const std::string& text) const {
        auto alphabet = initialAlphabet();
        std::vector<int> codes;
        codes.reserve(text.size());
```

```
        for (char ch : text) {
            const auto it = std::find(alphabet.begin(), alphabet.end(), ch);
            const int position = static_cast<int>(it - alphabet.begin());
            codes.push_back(position);
            moveToFront(alphabet, position);
        }
        return codes;
    }

    std::string decode(const std::vector<int>& codes) const {
        auto alphabet = initialAlphabet();
        std::string text;
        text.reserve(codes.size());
        for (int position : codes) {
            text.push_back(alphabet[position]);
            moveToFront(alphabet, position);
        }
        return text;
    }

private:
    static std::array<char, 27> initialAlphabet() {
        std::array<char, 27> alphabet{};
        alphabet[0] = '$';
        for (int i = 1; i <= 26; ++i) {
            alphabet[i] = static_cast<char>('a' + i - 1);
        }
        return alphabet;
    }

    static void moveToFront(std::array<char, 27>& alphabet, int position) {
        const char ch = alphabet[position];
        for (int i = position; i > 0; --i) {
            alphabet[i] = alphabet[i - 1];
        }
        alphabet[0] = ch;
    }
};

struct Run {
    int count;
    int value;
};

class RunLengthEncoding {
public:
    std::vector<Run> encode(const std::vector<int>& codes) const {
        std::vector<Run> runs;
        for (int value : codes) {
            if (!runs.empty() && runs.back().value == value) {
                ++runs.back().count;
            } else {
                runs.push_back({1, value});
            }
        }
        return runs;
    }

    std::vector<int> decode(const std::vector<Run>& runs) const {
```

```
        std::size_t size = 0;
        for (const Run& run : runs) {
            size += static_cast<std::size_t>(run.count);
        }
        std::vector<int> codes;
        codes.reserve(size);
        for (const Run& run : runs) {
            codes.insert(codes.end(), run.count, run.value);
        }
        return codes;
    }
};

class TextCompressor {
public:
    std::vector<Run> compress(const std::string& text) const {
        return rle_.encode(mtf_.encode(bwt_.encode(text)));
    }

    std::string decompress(const std::vector<Run>& runs) const {
        return bwt_.decode(mtf_.decode(rle_.decode(runs)));
    }

private:
    BurrowsWheelerTransform bwt_;
    MoveToFrontTransform mtf_;
    RunLengthEncoding rle_;
};

int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    std::string command;
    if (!(std::cin >> command)) {
        return 0;
    }
    const TextCompressor compressor;
    if (command == "compress") {
        std::string text;
        std::cin >> text;
        for (const Run& run : compressor.compress(text)) {
            std::cout << run.count << ' ' << run.value << '\n';
        }
    } else if (command == "decompress") {
        std::vector<Run> runs;
        int count;
        int value;
        while (std::cin >> count >> value) {
            runs.push_back({count, value});
        }
        std::cout << compressor.decompress(runs) << '\n';
    }
}
```

Экспериментальное исследование

Вместо перечня отдельных входных и выходных примеров выполнен воспроизводимый бенчмарк. Исследуются время отдельных преобразований, время полной цепочки, пиковая резидентная память процесса и фактический размер текстового результата RLE. Всего подготовлено 25 наборов: пять типов строк и пять длин.

Среда выполнения

Дата измерений: 2026-11-05. Процессор: AMD Ryzen 5 7430U with Radeon Graphics. Операционная система: Linux-7.0.0-38-generic-x86_64-with-glibc2.43. Доступная физическая память по /proc/meminfo — 14.50 ГиБ. Компилятор: g++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0. Параметры: -std=c++23 -O2 -DNDEBUG.

Фактическая локальная версия GCC отличается от указанной в проверяющей системе GCC 14.1; численные результаты относятся к приведённой среде. Бенчмарк выполнялся одним процессом последовательно. Частота процессора и фоновая нагрузка не фиксировались.

Наборы данных

Размеры исходного текста: 1 000, 5 000, 10 000, 50 000 и 100 000 символов. Все строки содержат только строчные латинские буквы. Для воспроизводимости псевдослучайных наборов начальное состояние генератора задаётся как 8062026 + длина строки.

Обозначение	Способ построения
Одинаковые	Повторение одной буквы a.
Период	Повторение abracadabra с обрезкой до заданной длины.
Случайные, 2	Независимый равновероятный выбор из a и b.
Случайные, 26	Независимый равновероятный выбор из a–z.
Блоки	Случайная буква a–z повторяется от 1 до 100 раз; блоки соединяются и обрезаются до нужной длины.

Одинаковые и периодические строки проверяют поведение на длинных общих префиксах. Случайные строки показывают работу с разной мощностью используемого алфавита. Блочная структура позволяет оценить влияние локальных повторений на MTF и RLE.

Методика измерений

Измерительный модуль benchmarks/benchmark.cpp подключает main.cpp с переименованием функции main. Он вызывает существующие классы без копирования алгоритмов. Для отдельных этапов заранее подготовлены промежуточные BWT, MTF и RLE; подготовка не входит во время операции.

Время измеряется std::chrono::steady_clock. Число повторений удваивается, пока пакет не займёт не менее 20 мс. Калибровка одновременно прогревает код и память. Затем выполняются пять пакетов одинакового размера. Время пакета делится на число операций; приводится медиана пяти средних времён. Сырые отсчёты сохранены отдельно.

Измеряются шесть преобразований и две полные цепочки. Выделение памяти и уничтожение результата входят в замер. Запуск процесса, чтение входа и форматирование вывода не входят. Размер возвращённого результата записывается в volatile-переменную, чтобы использование результата оставалось наблюдаемым.

Дополнительно консольная программа запускается трижды в каждом режиме для каждого набора. Монотонный таймер измеряет полное настенное время запуска через /usr/bin/time: с обёрткой, запуском процесса и вводом-выводом. Эту метрику следует отличать от времени алгоритма в памяти.

Пиковый RSS измеряется GNU time по формату %M в КиБ. Приводится максимум трёх запусков. RSS включает код, библиотеки и память дочерней программы, но не процесса подготовки данных. Это память процесса, а не точный объём рабочих массивов.

Размер результата — байты вывода compress, включая пробелы и переводы строк, без команды режима. Отношение S/m сравнивает его с размером исходного текста без перевода строки: меньше 1 означает уменьшение, больше 1 — увеличение.

Для каждого набора проверяется полное восстановление. В консольных запусках сжатый вывод сравнивается с заранее полученным результатом, а восстановленный текст — с оригиналом. Корректность контролируется при замерах; вместо списков входов и выходов приводятся характеристики производительности.

Время полной цепочки в памяти

Время указано в миллисекундах. C — compress, D — decompress. Каждая ячейка содержит медиану пяти пакетных измерений; входные строки заранее находятся в памяти.

Одинаковые

Длина m	C, мс	D, мс	C / (n $\log_2$ n), нс
1 000	0.1254	0.0100	12.57
5 000	0.7616	0.0482	12.39
10 000	1.6389	0.0963	12.33
50 000	10.0962	0.6320	12.94
100 000	21.4221	1.3377	12.90

Период

Длина m	C, мс	D, мс	C / (n $\log_2$ n), нс
1 000	0.1182	0.0095	11.85
5 000	0.7517	0.0488	12.23
10 000	1.6264	0.0956	12.24
50 000	9.9431	0.6329	12.74
100 000	21.1518	1.3028	12.73

Случайные, 2

Длина m	C, мс	D, мс	C / (n $\log_2$ n), нс
1 000	0.0636	0.0114	6.38
5 000	0.3412	0.0627	5.55
10 000	0.7282	0.1442	5.48
50 000	4.4335	0.8065	5.68
100 000	10.2394	1.8286	6.16

Время полной цепочки (продолжение)

Случайные, 26

Длина m	C, мс	D, мс	C / (n $\log_2$ n), нс
1 000	0.0385	0.0206	3.86
5 000	0.2407	0.1185	3.92
10 000	0.5230	0.2572	3.94
50 000	3.3212	1.3207	4.26
100 000	7.2119	2.8067	4.34

Блоки

Длина m	C, мс	D, мс	C / (n $\log_2$ n), нс
1 000	0.0850	0.0098	8.52
5 000	0.4714	0.0495	7.67
10 000	0.8786	0.1000	6.61
50 000	5.2186	0.6883	6.69
100 000	10.9583	1.5225	6.60

Время отдельных этапов при m = 100 000

В таблицах приведены медианы в миллисекундах. Этапы измерялись независимо. Их сумма может отличаться от времени полной цепочки из-за состояния кэша, выделения памяти и вариаций нагрузки.

Прямые преобразования

Тип строки	BWT	MTF	RLE	Вся цепочка
Одинаковые	20.3317	0.9238	0.1367	21.4221
Период	20.5031	0.9238	0.1746	21.1518
Случайные, 2	9.1406	1.3150	0.8047	10.2394
Случайные, 26	4.7921	1.6696	0.8733	7.2119
Блоки	10.0448	0.9486	0.1491	10.9583

Обратные преобразования

Тип строки	RLE	MTF	BWT	Вся цепочка
Одинаковые	0.0322	0.4038	0.5501	1.3377
Период	0.0320	0.4032	0.5561	1.3028
Случайные, 2	0.4704	0.5718	0.8061	1.8286
Случайные, 26	0.7291	0.8082	0.7069	2.8067
Блоки	0.0736	0.4036	0.6928	1.5225

Разброс измерений полной цепочки

Тип строки	C min–max, мс	D min–max, мс
Одинаковые	21.121–21.492	1.323–1.381
Период	20.789–21.512	1.296–1.305
Случайные, 2	9.813–10.632	1.815–1.841
Случайные, 26	7.099–7.362	2.795–2.842
Блоки	10.805–10.974	1.512–1.536

Различия между типами данных обусловлены числом шагов удвоения, доступом по LF-переходам, текущими позициями символов MTF и количеством серий RLE. Быстрая остановка BWT на различных классах возможна раньше достижения полного размера строки; поэтому верхняя оценка O(n log n) не требует одинакового времени на всех входах одной длины.

Размер результата и влияние структуры текста

При m = 100 000 вход содержит ровно 100 000 байтов. Число серий r равно числу строк вывода RLE. Доля нулей вычислена среди n = m + 1 кодов MTF; повторение кода не обязательно означает повторение исходной буквы.

Тип строки	Серии r	Нули, %	Выход, байт	S/m
Одинаковые	3	100.00	16	0.0002
Период	14	99.99	80	0.0008
Случайные, 2	50064	49.88	200347	2.0035
Случайные, 26	96087	3.86	443516	4.4352
Блоки	5372	96.30	24380	0.2438

Изменение отношения S/m при росте длины

Длина	Одинаковые	Период	Случайные, 2	Случайные, 26	Блоки
1000	0.0140	0.0660	1.9350	4.4650	0.2190
5000	0.0030	0.0168	1.9990	4.4160	0.2470
10000	0.0015	0.0081	1.9792	4.4207	0.2460
50000	0.0003	0.0017	2.0020	4.4402	0.2498
100000	0.0002	0.0008	2.0035	4.4352	0.2438

На повторяющихся данных BWT и MTF создают длинные серии, поэтому число пар растёт существенно медленнее длины текста. На случайных строках серий больше, а запись каждой пары требует как минимум двух чисел, пробела и перевода строки. Дополнительного энтропийного кодирования в условии нет, поэтому значительное увеличение текстового представления случайных данных является ожидаемым свойством этой цепочки.

Даже высокая доля нулей сама по себе не определяет размер результата: важны длины непрерывных серий и число их разделителей. Метрика размера учитывает реальный формат вывода задачи, а не гипотетическую упаковку кодов в фиксированное число бит.

Полный запуск и пиковая память

Результаты для максимальной длины m = 100 000. Время — медиана трёх запусков с вводом-выводом и обёрткой GNU time; память — максимальный RSS среди этих запусков. Мебибайты получены делением КиБ на 1024.

Тип строки	C, мс	D, мс	RSS C, МиБ	RSS D, МиБ
Одинаковые	26.62	6.02	5.76	4.56
Период	26.12	5.56	5.73	4.56
Случайные, 2	19.04	10.31	5.68	4.93
Случайные, 26	18.86	14.23	5.72	5.34
Блоки	18.22	6.83	5.75	4.68

Масштабирование RSS на случайном алфавите из 26 букв

Длина m	RSS сжатия, МиБ	RSS восстановления, МиБ
1000	3.99	4.03
5000	4.16	4.03
10000	4.25	4.06
50000	4.58	4.43
100000	5.72	5.34

На максимальной длине наибольшее медианное время полного сжатия среди исследованных типов составило 26.62 мс, восстановления — 14.23 мс. Наибольший измеренный RSS по всем наборам и режимам составил 5.76 МиБ. В этой локальной среде результаты находятся ниже ограничений 1 с и 256 МБ.

Эти измерения не являются гарантией времени на другом процессоре или в проверяющей системе. Накладные расходы запуска особенно заметны на малых входах; RSS также имеет постоянную составляющую и меняется по страницам памяти. Поэтому небольшие колебания RSS не опровергают линейную асимптотическую оценку.

Итоги экспериментального исследования

Выполнено 1000 пакетных измерений: 25 наборов данных × 8 операций × 5 пакетов. Дополнительно проведено 150 полных запусков программы: 25 наборов × 2 режима × 3 повтора. Для каждого набора подтверждено точное восстановление исходного текста.

Сырые результаты измерений сохранены в таблицах stages.csv и cli.csv. Они содержат параметры входных данных, число повторений, время операций и пиковую резидентную память. Сводные значения рассчитаны по этим измерениям: для времени использована медиана, для RSS — максимум повторных запусков.

Вывод

В курсовом проекте реализована цепочка BWT+MTF+RLE для сжатия и восстановления строк из строчных латинских букв. Прямое BWT строится сортировкой циклических сдвигов методом удвоения с сортировкой подсчётом; обратное преобразование использует LF-переходы. MTF адаптирует позиции символов к текущему контексту, RLE объединяет серии одинаковых числовых кодов.

Корректность обоснована для каждого преобразования и их композиции. Сжатие требует O(n log n) времени, восстановление — O(n), память обоих режимов — O(n) при фиксированном алфавите. Алгоритмы выделены в отдельные классы и объединены композицией, а консольный ввод-вывод отделён от вычислений.

Бенчмарк на 25 наборах показал зависимость скорости и размера результата от структуры текста. Повторяющиеся данные эффективно представляются сериями, тогда как на случайном тексте итоговая текстовая запись может увеличиваться. Полное восстановление всех исследованных строк подтверждено во время измерений. Измеренные время и память удовлетворяют ограничениям задачи в указанной локальной среде.